Milestone 2: Predicting Diabetes Using Machine Learning

1. Problem Statement / Description

This project focuses on predicting diabetes using the Diabetes_012 dataset, which contains over 250,000 survey responses with health-related, behavioral, and demographic information. Each row represents an individual, and the target variable (Diabetes_012) indicates whether that person is non-diabetic (class 0), pre-diabetic (class 1), or diabetic (class 2). The dataset is well suited for classification because it includes many meaningful predictors such as body mass index (BMI), physical activity, fruit and vegetable consumption, smoking behavior, sleep duration, and general health indicators.

The main goal of the project is to build machine-learning models that can identify patterns associated with higher diabetes risk and classify individuals based on the likelihood of having diabetes. By applying different models—such as K-Nearest Neighbors (KNN), Logistic Regression, Decision Trees, Gradient Boosting, and several clustering methods, I aim to understand which features most influential and which modeling techniques are provide the strongest predictive performance on this large, imbalanced health dataset.

2. Potential Impact

This project has important real-world implications. Diabetes affects millions of people worldwide, and early detection can significantly reduce long-term complications and healthcare costs. A well-designed predictive model:

- It can help healthcare providers flag high-risk individuals earlier.
- Can support public-health decision making by identifying patterns in lifestyle and demographic factors.
- Can help design targeted prevention programs for communities with elevated diabetes risk.
- Demonstrates how data science can directly support disease prevention and health-equity efforts.

Overall, this project shows how machine-learning techniques can transform large amounts of health data into insights that support earlier interventions and more informed public-health strategies.

3. Exploratory Data Analysis (EDA)

Before building models, I performed Exploratory Data Analysis (EDA) to better understand the structure of the Diabetes_012 dataset. One of the first observations was that the classes are highly imbalanced: most samples belong to class 0 (no diabetes), while classes 1 and 2 contain far fewer individuals. This imbalance indicated that accuracy alone would not be a reliable

evaluation metric. Consequently, I planned to rely more on weighted F1-scores and confusion matrices when evaluating models.

I examined the distributions of several key features, including BMI, general health (GenHlth), physical activity, and age category. The BMI distribution was right-skewed, with many individuals in normal or overweight ranges and a smaller tail of individuals with very high BMI values. Features such as physical activity and fruit/vegetable intake showed that many respondents reported low levels of healthy behavior. These patterns suggested that lifestyle factors would be important predictors in the models.

I also computed a correlation matrix for the numeric features. Most pairwise correlations were relatively weak, which is common in survey data, but some combinations—such as BMI and general health—showed modest correlations. This supported the idea that linear models might not fully capture the relationships in the data, while tree-based models and ensembles could take advantage of non-linear interactions between features.

Because the features had very different scales (for example, BMI versus binary indicators), EDA confirmed that scaling would be essential for distance-based models like KNN. It also motivated the use of metrics that are robust to class imbalance and the exploration of both supervised and unsupervised techniques.

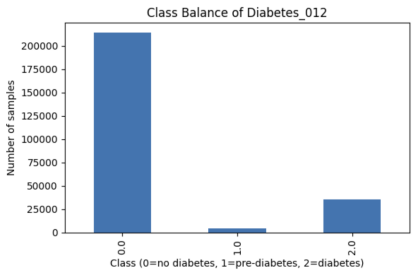

Figure 1. Class balance bar chart for the Diabetes_012 target variable.

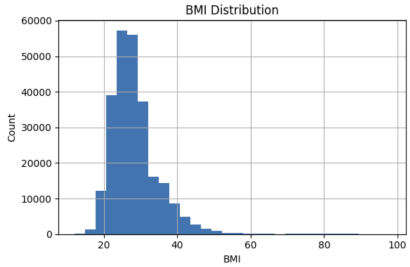

Figure 2. Histogram of BMI values in the dataset.

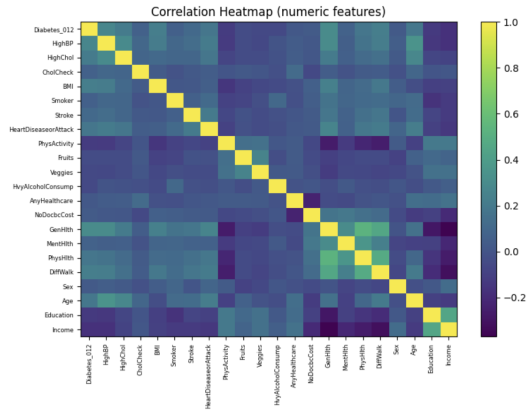

Figure 3. Correlation heatmap for numeric features.

4. Week 8: K-Nearest Neighbors (KNN) Modeling

In this part of the project, I used the K-Nearest Neighbors (KNN) classifier to predict diabetes. Because KNN makes predictions based on distances between samples, I first standardized the features, so they were on a comparable scale. This step was important because some features, such as BMI, have much larger numeric ranges than others. Without scaling, KNN would give too much influence to features with large values and too little influence on binary or small-scale features.

After scaling, I trained a baseline KNN model with k = 5 neighbors. This model achieved a training accuracy of approximately 0.869 and a test accuracy of approximately 0.832, along with a weighted F1-score of about 0.803. These values suggested that the model generalized reasonably well but also showed a small gap between training and test performance, indicating mild overfitting.

To study overfitting more closely, I evaluated KNN with k values from 1 to 15. When k was very small, especially at k = 1, the model nearly memorized the training data, achieving a very high training accuracy around 0.992 while the test accuracy dropped to about 0.780. This is a textbook example of overfitting: the model fits noise and specific details in the training set that do not generalize to new data. As k increased, the gap between training and test accuracy shrank. For k values between roughly 11 and 15, the train and test accuracies stabilized near 0.84, which indicates much healthier generalization.

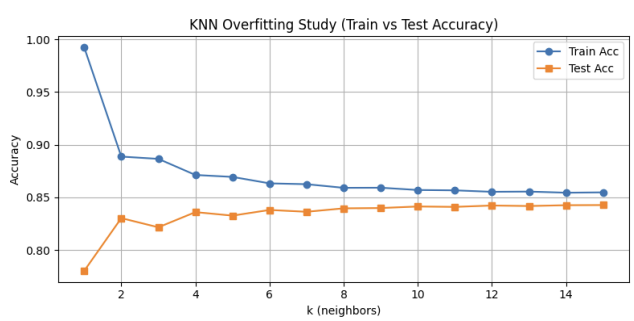

Figure 4. KNN training and test accuracy as a function of k.

Based on these results, I selected k = 13 as a final KNN configuration. At k = 13, the test accuracy was approximately 0.842 and the weighted F1-score was around 0.801. The confusion matrix for this model showed that class 0 (no diabetes) was predicted very well, while classes 1 and 2 remained more difficult to classify because they had far fewer samples. This behavior is consistent with the class imbalance observed in EDA.

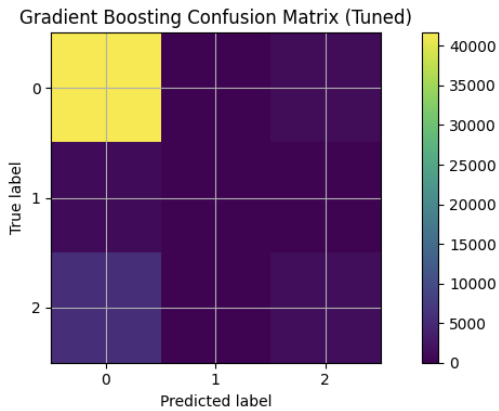

Figure 5. Confusion matrix for the final KNN model with k = 13.

For KNN, I used accuracy, weighted F1-score, and confusion matrices as the main evaluation metrics. Accuracy measured the overall fraction of correct predictions, but weighted F1-score was more informative in the presence of class imbalance because it accounts for both precision and recall for each class, weighted by class frequency. I also used GridSearchCV to tune k, the distance metric (Euclidean versus Manhattan), and whether to use uniform or distance-based weighting for neighbors. This tuning process helped select a value of k and weighting scheme that improved generalization and slightly increased performance on the minority classes.

Overall, KNN behaved as expected: small k values produced high variance and overfitting, while larger k values produced smoother decision boundaries and better generalization. With proper scaling and tuning, KNN served as a strong and interpretable baseline model for diabetes prediction.

5. Week 9: Gradient Boosting Modeling

Gradient Boosting is a powerful ensemble method that builds many small decision trees (weak learners) sequentially, where each tree attempts to correct the errors of the previous ones. In this project, I used Gradient Boosting to improve predictive performance beyond what was achieved with KNN and individual decision trees. Because Gradient Boosting is a flexible and expressive model, it can easily overfit if the trees are too deep or if too many trees are added without constraints.

To reduce overfitting, I restricted each tree to a small depth (max_depth = 2 or 3) and studied how the model behaved as I changed the number of trees (n_estimators). The baseline Gradient

Boosting model, with about 100 trees, already showed balanced behavior, with both training and test accuracy around 0.849. This contrasted with KNN at small k values, where training accuracy was much higher than test accuracy.

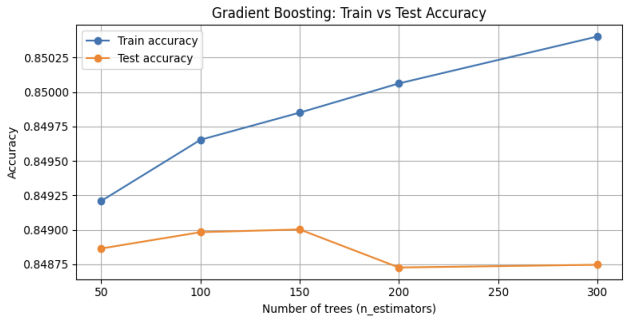

Figure 6. Gradient Boosting training and test accuracy for different numbers of trees.

I then evaluated Gradient Boosting for several n_estimators values, such as 50, 100, 150, 200, and 300, while keeping a moderate learning rate (0.1) and shallow trees. As the number of trees increased, training accuracy continued to improve slightly, but test accuracy plateaued and did not benefit much beyond about 150–200 trees. This pattern suggested that adding more trees primarily helped the model fit the training data more closely, with diminishing returns for generalization and potential risk of overfitting.

To further refine the model, I used GridSearchCV to tune three main hyperparameters: learning_rate, max_depth, and n_estimators. The best combination was a learning_rate of 0.1, max_depth of 2, and n_estimators of about 200. With these settings, the final Gradient Boosting model achieved a test accuracy of approximately 0.849 and a weighted F1-score around 0.813. The confusion matrix for this tuned model again showed excellent performance in class 0, with more modest performance on classes 1 and 2.

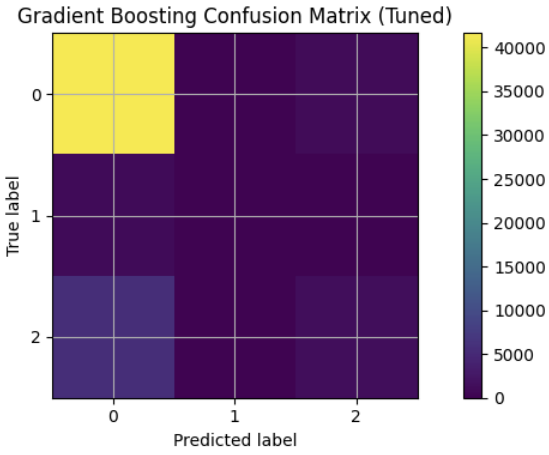

Figure 7. Confusion matrix for the tuned Gradient Boosting model.

For Gradient Boosting, I used the same evaluation metrics as for KNN: accuracy, weighted F1-score, and confusion matrices. Weighted F1-score was particularly important because it revealed how well the model handled minority classes. Although class imbalance remained a challenge, Gradient Boosting provided the strongest and most stable performance among all supervised models tested in this milestone.

6. Week 10: K-Means Clustering and Silhouette Score

In addition to supervised learning, I applied K-Means clustering to explore natural groupings in the Diabetes_012 dataset without using the diabetes labels. Before clustering, I scaled the features and used Principal Component Analysis (PCA) to reduce the data to two principal components. This step made K-Means faster and allowed for clear visualization of clusters in two dimensions.

I evaluated K-Means for k values between 2 and 8 and used both the elbow method and the silhouette score to select a good number of clusters. The elbow curve showed a clear bend around k = 3, where the decrease in within-cluster sum of squared errors (SSE) began to slow. The silhouette scores were highest for k = 2 but remained reasonable for k = 3. Combining these criteria, I selected k = 3 as a good balance between model simplicity and cluster interpretability.

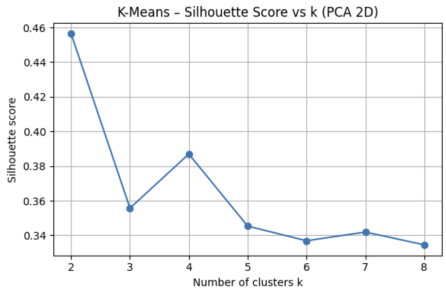

Figure 8. K-Means elbow curve and silhouette scores for different k values.

The PCA scatter plot with k = 3 showed three distinct but overlapping groups in the transformed feature space. Although these clusters do not align perfectly with the diabetes classes, they suggest that individuals can be grouped into a small number of health-behavior profiles. This unsupervised perspective complements the supervised models by revealing structure in the data that does not depend directly on the label.

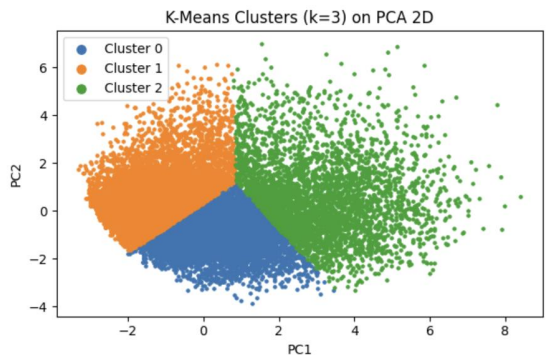

Figure 9. PCA scatters plot of K-Means clusters with k = 3.

7. Week 11: DBSCAN Clustering

I also applied Density-Based Spatial Clustering of Applications with Noise (DBSCAN) to the PCA-transformed data. DBSCAN is different from K-Means because it does not require specifying the number of clusters in advance and can find clusters of arbitrary shapes while labeling isolated points as noise. It is controlled mainly by two parameters: eps (the neighborhood radius) and MinPts (the minimum number of points needed to form a dense region).

To choose a good eps value, I created a k-distance plot based on the distance to the nearest 10th neighbor. The plot was mostly flat at small distances and then curved upward around distances of approximately 0.1–0.15. This elbow suggested that eps values in this range would separate dense regions from sparser areas. Using an eps value of about 0.12 and MinPts = 10, DBSCAN identified several clusters and a set of noise points.

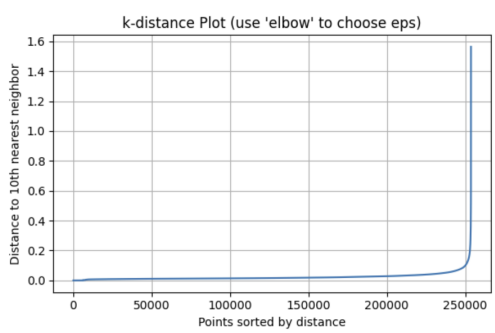

Figure 10. k-distance plot used to select eps for DBSCAN.

The DBSCAN cluster plot in PCA space showed compact core regions with surrounding border points, as well as noise points that did not belong to any cluster. This behavior is useful for identifying outliers—individuals whose health and behavior profiles are unusual compared to the main population. While the DBSCAN clusters do not directly optimize prediction performance, they provide additional insight into the density structure of the data and highlight atypical patterns that might be important in future analysis.

8. Overall Modeling Reflections: Overfitting, Metrics, and Tuning

Across all models, I used several techniques to avoid overfitting and improve generalization. For KNN, the main lever was the choice of k: very small k values led to high variance and overfitting, while larger k values smoothed the decision boundary and reduced sensitivity to noise. For Gradient Boosting, I controlled complexity by limiting the depth of each tree, using a moderate learning rate, and tuning the number of trees. These steps prevented the model from fitting the training data perfectly and helped keep train and test performance closely aligned.

I relied on multiple evaluation metrics. Accuracy measured overall correctness, but because the dataset is heavily imbalanced, I placed more emphasis on weighted F1-scores and confusion matrices. Weighted F1-score balances precision and recall across all classes and accounts for class frequencies, making it more informative than accuracy alone. For clustering models, I used the silhouette score to assess cluster quality and the elbow method and k-distance plot to select appropriate hyperparameters such as k and eps.

Hyperparameter tuning played a key role in improving performance. I used GridSearchCV to systematically explore values of k for KNN and to tune learning_rate, max_depth, and n_estimators for Gradient Boosting. For clustering, I tuned k in K-Means using elbow and silhouette criteria and tuned eps and MinPts for DBSCAN using the k-distance plot and visual inspection of resulting clusters. Some results were expected—for example, Gradient Boosting outperformed simpler models—while others highlighted persistent challenges, such as the difficulty of accurately predicting minority classes despite tuning.

9. Limitations

There are several limitations to this analysis. First, the Diabetes_012 dataset is strongly imbalanced, with class 0 containing most of the samples. This makes it hard for models to learn patterns for classes 1 and 2 and leads to lower recall for those classes. Second, the dataset is based on self-reported survey responses, which can include measurement error, recall bias, or inconsistent reporting. These issues introduce noise that can reduce model performance. Third, some of the clustering results depend on PCA, which reduces dimensionality but also discards information; different projections might produce different cluster shapes. Finally, the models were trained on a single year of BRFSS data (2015), so their performance may not generalize perfectly to other years or populations without retraining and validation.

10. Final Summary

In this milestone, I used a combination of supervised and unsupervised learning techniques to analyze the Diabetes_012 dataset and predict diabetes status. KNN provided a strong baseline after scaling and tuning, while Gradient Boosting delivered the best overall predictive performance with carefully chosen three depth, learning rate, and number of estimators. Logistic

Regression and Decision Trees, though not discussed in as much detail, offered additional perspectives on interpretability and model flexibility.

Clustering methods such as K-Means and DBSCAN complemented the supervised models by revealing natural groupings and density patterns in the data, as well as highlighting potential outliers. Across all approaches, class imbalance remained a core challenge, especially for accurately predicting the pre-diabetes and diabetes classes. Nonetheless, the combination of EDA, supervised learning, and clustering provided a rich and multifaceted view of how health behaviors and demographics relate to diabetes risk.

Overall, this milestone demonstrates how machine-learning methods can be applied to large public-health datasets to support early risk identification, guide preventive interventions, and deepen our understanding of the factors that contribute to chronic diseases like diabetes.

11. References

Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research, 12*, 2825–2830. https://www.researchgate.net/publication/51969319_Scikit-learn_Machine_Learning_in_Python

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232. https://www.researchgate.net/publication/2424824_Greedy_Function_Approximation_A_Gradient_Boosting_Machine

StatQuest with Josh Starmer. (n.d.). *StatQuest: Statistics, machine learning, and more!* YouTube. https://www.youtube.com/user/joshstarmer

Zhang, T. (2017). Nearest neighbor classification and the bias-variance tradeoff. *Journal of Machine Learning Research*. https://pmc.ncbi.nlm.nih.gov/articles/PMC4916348/